Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Робототехника и комплексная автоматизация»

КАФЕДРА «Компьютерные системы автоматизации производства»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ

НА ТЕМУ:

«Проектирование программной части сервера устройства для облачной системы управления конвейером на основе облачной платформы Yandex.Cloud»

Студент ____РК9-74Б___ _____21.12.2020__ _Е. И. Рындя____
(Группа) (Подпись, дата) (И.О.Фамилия)

Руководитель курсового проекта _____21.12.2020_____ _С. А. Буханов_____
(Подпись, дата) (И.О.Фамилия)

Консультант _____ _М. В. Овсянников_
(Подпись, дата) (И.О.Фамилия)

2020 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой _РК9_____
(Индекс)
_____ _С. С. Гаврюшин_
(И.О.Фамилия)
« _____ » _____ 2020 г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине __Основы автоматизации производства_____

Студент группы ____РК9-74Б____

_____Рындя Егор Игоревич_____
(Фамилия, имя, отчество)

Тема курсового проекта __Проектирование программной части сервера устройства_____
для облачной системы управления конвейером на основе облачной платформы Yandex.Cloud
_____
_____
_____

Направленность КП (учебный, исследовательский, практический, производственный, др.)
_____учебный_____
Источник тематики (кафедра, предприятие, НИР) _____кафедра_____

График выполнения проекта: 25% к 5 нед., 50% к 9 нед., 75% к 12 нед., 100% к 16 нед.

Задание

1) Проанализировать существующее программное обеспечение, реализованное Denford, и выявить его недостатки;
2) На основе анализа полученных данных сформулировать требования, предъявляемые к проектируемой программной части сервера устройства;
3) Разработать алгоритмы управления конвейером Denford для получения им команд от облачной системы Yandex.Cloud;
4) Разработать программный код на языке Python для программной части сервера устройства.

Оформление курсового проекта:

Расчетно-пояснительная записка на 36 листах формата А4.
Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.): 22 рисунков, 15 слайдов.
Дата выдачи задания «19» сентября 2020 г.

Руководитель курсового проекта _____21.12.2020___ _С. А. Буханов____
(Подпись, дата) (И.О.Фамилия)

Студент _____21.12.2020_ _Е. И. Рындя_____
(Подпись, дата) (И.О.Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

РЕФЕРАТ

Курсовой проект выполнен по теме: «Проектирование программной части сервера устройства для облачной системы управления конвейером на основе облачной платформы Yandex Cloud».

Целью выполнения курсового проекта является создание программной части сервера устройства для облачной системы управления модулем транспортно-накопительной системы (конвейером) на основе облачной платформы «Yandex Cloud».

Курсовой проект состоит из расчётно-пояснительной записки, которая содержит 36 листа формата А4, 22 рисунков, 10 используемых источников, 2 приложения; презентации, состоящей из 15 слайдов.

Ключевые слова: сервер устройства, облачная платформа, интернет вещей, промышленный интернет вещей, «Yandex Cloud».

Данная работа состоит из шести разделов.

В первом разделе приведен анализ существующей системы, по результатам которого были выделены недостатки.

Во втором разделе представлен анализ кольцевого ленточного конвейера и его задач в ГПС.

В третьем разделе описано взаимодействие сервер устройства с кольцевым ленточным конвейером.

В четвертом разделе описано взаимодействие между облачной платформой и сервером устройства.

В пятом разделе разработаны алгоритмы работы основных функций программы.

В шестом разделе рассматривается случай аварийной остановки ГПС.

Оглавление

Основные термины 4
Введение 5
1 Анализ гибкой производственной системы «Denford». 8
2 Анализ кольцевого ленточного конвейера и его задач в ГПС 8
3 Взаимодействие сервер устройства с кольцевым ленточным конвейером. 11
4 Взаимодействие сервер устройства с облачной платформой «Yandex Cloud» ... 13
5 Разработка программной части сервера устройства 13
5.1 Использование многопоточности для одновременного управления замками кольцевого ленточного конвейера 15
5.2 Валидация полученных команд от облачной платформы. 16
5.3 Проектирование функции, обрабатывающей логику открытия любого замка. 17
5.4 Проектирование функции, обрабатывающей логику работы Замка 0. 18
5.5 Проектирование функции, обрабатывающей логику работы Замка 1. 19
5.6 Проектирование функции, обрабатывающей логику работы Замка 2 22
5.7 Проектирование функции, обрабатывающей логику работы Замка 3 23
6 Аварийная остановка 24
Заключение 22
Список используемой литературы 23
Приложение 1. Код функции обработки и выполнения задания на языке Python 30
Приложение 2. Слайды из презентации..... 237

Основные термины

ГПС - совокупность в разных сочетаниях оборудования с ЧПУ, робототизированных технологических комплексов, гибких производственных модулей, отдельных единиц технологического оборудования и систем обеспечения их функционирования в автоматическом режиме в течение заданного интервале времени, обладающую свойством автоматизированной переналадки при производстве изделий произвольной номенклатуры в установленных пределах значений их характеристик.

Raspberry Pi 3 Model B - полноценный одноплатный компьютер размером с банковскую карту.

Замок - устройство представляющее собой электронную защелку со встроенным электромагнитным датчиком, срабатывающем на направляющую часть паллеты. В данном курсовом проекте используется замок для ленточного конвейера WPS-600, с помощью которого и реализуются алгоритмы автоматизированной работы конвейера.

Яндекс.Облако - набор связанных сервисов, облачная платформа от российской интернет-компании Яндекс.

Облачные технологии (облачные вычисления, cloud computing) – технологии распределенной обработки цифровых данных, с помощью которых компьютерные ресурсы предоставляются интернет-пользователю как онлайн-сервис.

Автоматизированная система управления технологическим процессом (АСУ ТП) - группа решений технических и программных средств, предназначенных для автоматизации управления технологическим оборудованием на промышленных предприятиях.

«MQTT-брокер» - сервер-посредник, используемый в протоколе MQTT.

Введение

Одной из важнейших тем в современной автоматизации является интернет вещей (Internet of things). Интернет вещей соединяет окружающие нас объекты в компьютерную сеть. Они обмениваются информацией между собой, работают без вмешательства человека и в режиме реального времени. Даже на нынешних ранних этапах Интернет вещей привел к тому, что изменилось взаимодействие между корпорациями, потребителями и окружающими предметами. С развитием данной технологии вскоре появился промышленный интернет вещей (Industrial Internet of Things) - система объединенных компьютерных сетей и подключенных промышленных (производственных) объектов со встроенными датчиками и ПО для сбора и обмена данными, с возможностью удаленного контроля и управления в автоматизированном режиме, без участия человека, так называемая – четвертая промышленная революция.

Преимущества IIot:

1) *Повышение эффективности* - самым большим преимуществом IIoT является то, что он дает производителям возможность автоматизировать и, следовательно, оптимизировать свою операционную эффективность. Робототехника и автоматизированное оборудование могут работать более эффективно и точно, повышая производительность и помогая производителям оптимизировать свои функции.
2) *Уменьшение количества ошибок* - промышленный Интернет вещей дает производителям возможность оцифровывать практически каждую часть своего бизнеса. За счет сокращения ручного процесса и записей производители могут снизить самый большой риск, связанный с ручным трудом, - человеческий фактор.

3) *Профилактическое обслуживание* - ничто так не влияет на производственные операции больше, чем простои оборудования. Когда производительность и функционирование оборудования постоянно отслеживаются. Эти данные дают компаниям информацию, необходимую для выявления любой проблемы до ее возникновения. Затем они могут запланировать техническое обслуживание до простоя.
4) *Повышение безопасности* - все данные и датчики, необходимые для полностью функционирующего производственного процесса IIoT, также помогают повысить безопасность на рабочем месте. В случае аварии все на предприятии могут быть предупреждены, операции могут быть прекращены, а руководство компании может вмешаться и убедиться, что авария и инцидент устранены. Этот инцидент также может генерировать ценные данные, которые могут помочь предотвратить его повторение в будущем.
5) *Снижение затрат* - все вышеупомянутые преимущества IIoT - профилактическое обслуживание, меньшее количество ошибок, улучшенный контроль качества и максимальная эффективность, а также анализ данных об операциях, производстве, маркетинге и продажах – все это способствует снижению затрат и увеличению прибыли производителя.

Параллельно в промышленности наметился переход от классических АСУ ТП к использованию Iiot. использованию промышленного интернета вещей. Если раньше структуру АСУ ТП можно было представить в форме классической пирамиды, где четко просматриваются четыре отдельных уровня (рисунок 1):

1) *Уровень планирования ресурсов (ERP)*;
2) *Уровень управления процессами (MES, SCADA)*;
3) *Уровень контроля (PLC, HMI)*;
4) *Физический уровень (датчики, приводы)*;

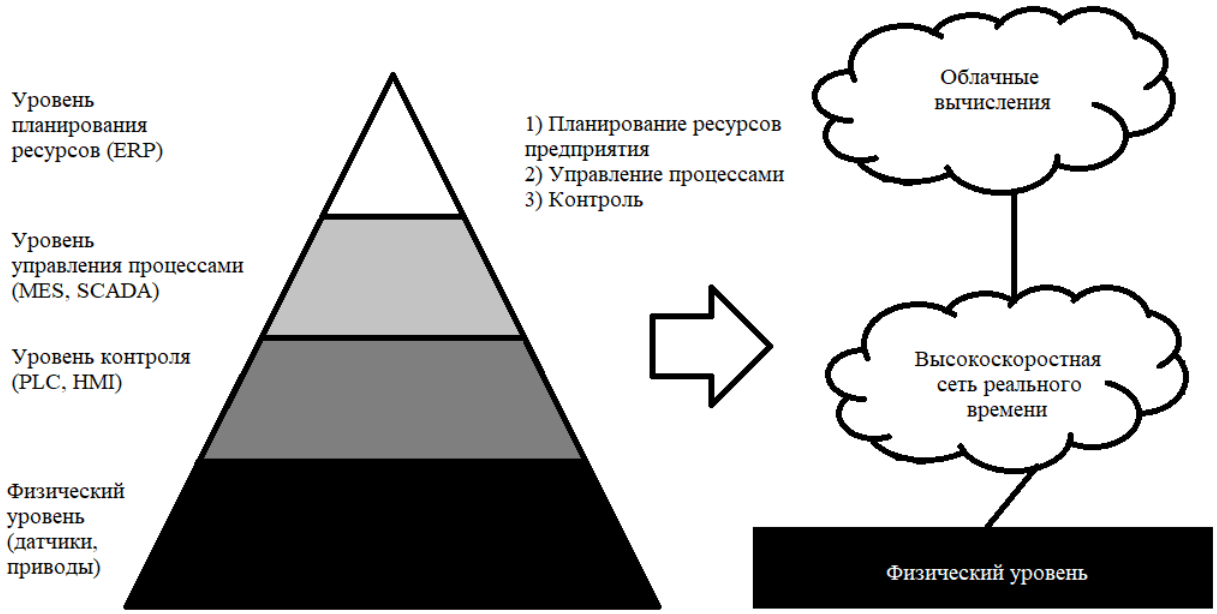


Рисунок 1. Переход от традиционных АСУ ТП к IIOT

Всё больше предприятий переходят к облачным вычислениям, когда планирование ресурсов предприятия, управление процессами, а также контроль осуществляются внутри облачной платформы, высокоскоростная сеть реального времени осуществляет связь между «облаком» и физическим уровнем

Темой данного проекта является автоматизация транспортно-накопительного модуля гибкой производственной системы «Denford» - кольцевого ленточного конвейера.

Цель курсового проекта – создание программной части сервера устройства для облачной системы управления кольцевым конвейером на основе облачной платформы «Yandex Cloud».

Задачи курсового проекта:

1) Проанализировать существующее программное обеспечение, реализованное Denford, и выявить его недостатки.

2) На основе проанализированных данных сформулировать требования, относящиеся к проектируемой программной части сервера устройства.

3) Разработать алгоритмы управления конвейером Denford для получения им команд от облачной системы «Yandex Cloud».

4) Разработать программный код на языке Python для программной части сервера устройства.

1. Анализ гибкой производственной системы «Denford».

В качестве исследуемой гибкой производственной системы была выбрана система производителя «Denford», а именно модуль её транспортно-накопительной системы – кольцевой ленточный конвейер. Структурная схема данной ГПС представлена на рисунке 2.

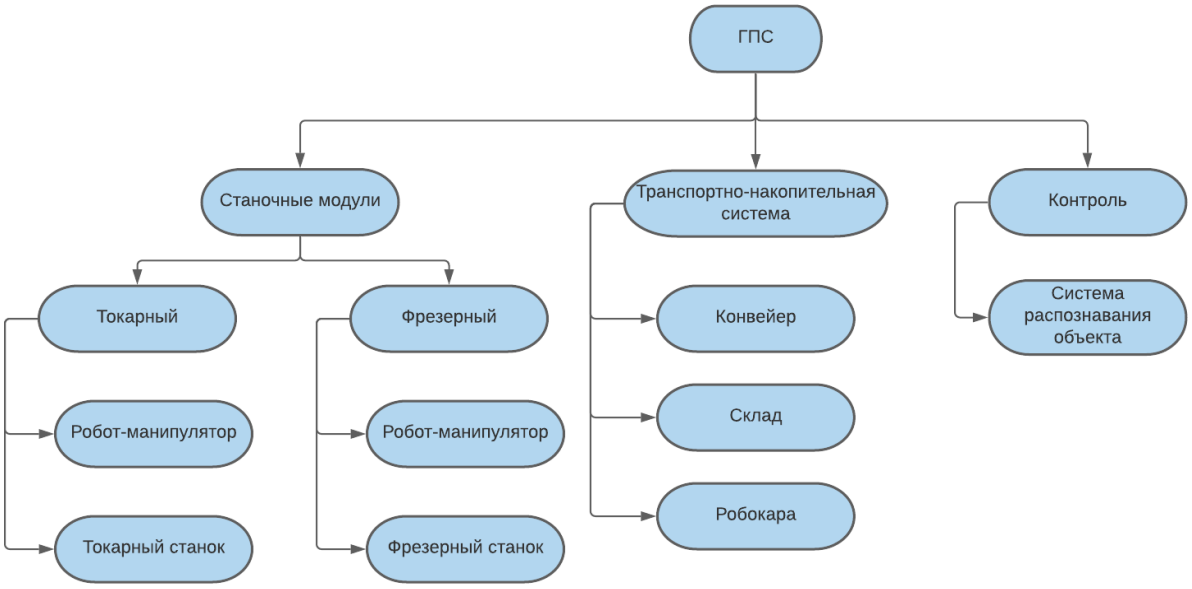


Рисунок 2. Структурная схема ГПС

2. Анализ кольцевого ленточного конвейера и его задач в ГПС.

В системе ГПС кольцевой ленточный конвейер взаимодействует со следующими элементами производственного процесса:

-Склад

-Производственный модуль с фрезерным станком

-Производственный модуль с токарным станком

-Модуль “Проверка качества”

Перед каждым из этих элементов на конвейере расположены специальные пневматические замки, позволяющие останавливать паллеты с заготовкой, передвигающиеся по ленте конвейера, также в каждом замке присутствует электромагнитный датчик, который определяет наличие паллеты в замке. Движение паллет осуществляется против часовой стрелки (рисунок 3). Стоит заметить, что в то время, как замки, соответствующие производственным модулям со станками и модулю “Проверка качества”, находятся прямо напротив соответствующих элементов, замок, предназначенный для отправки паллет на склад или для отправки паллеты на второй круг, находится на некотором расстоянии от места спуска штабелера, который и занимается отправкой паллет на склад.

Итак, стандартный цикл обслуживания конвейером элементов ГПС выглядит следующим образом:

1) Штабелер опускает паллету со склада;

2) Паллета прибывает на Замок 0;

3) Система замков конвейера приступает к выполнению конкретного сценария для данной паллеты;

4) После выполнения сценария паллета прибывает на Замок 3;

5) Там она ожидает прибытия штабелера, который отправит паллету обратно на склад, или отправляется на второй круг;

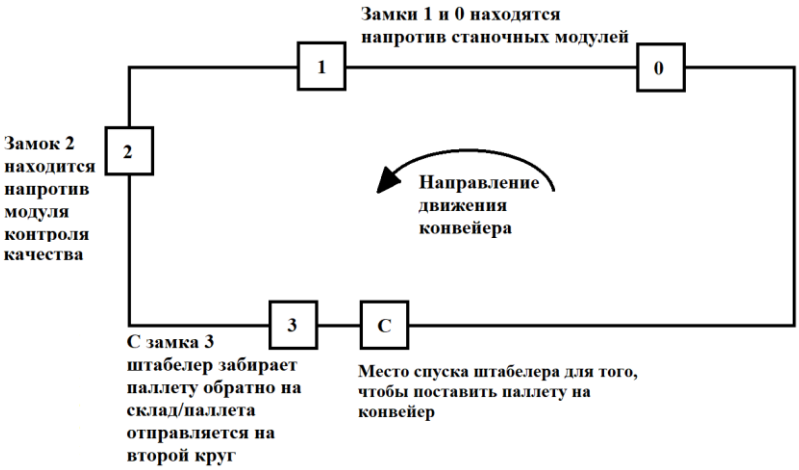


Рисунок 3. Схема расположения замков.

Из данного цикла следует, что все управление конвейером осуществляется посредством замков, которые могут находиться в двух положениях – открытом или нормально-закрытом. Задачи кольцевого ленточного конвейера заключаются в переключении данных пневматических замков, отправки и получения запросов о состоянии заготовок на соответствующих производственных модулях.

В существующей системе управление пневматическими замками осуществляется с помощью ЭВМ, подключенной по локальной сети (LAN) к 16-битной промышленной плате ввода-вывода, которая подключена к пневматическому контроллеру по параллельному порту (рисунок 4).

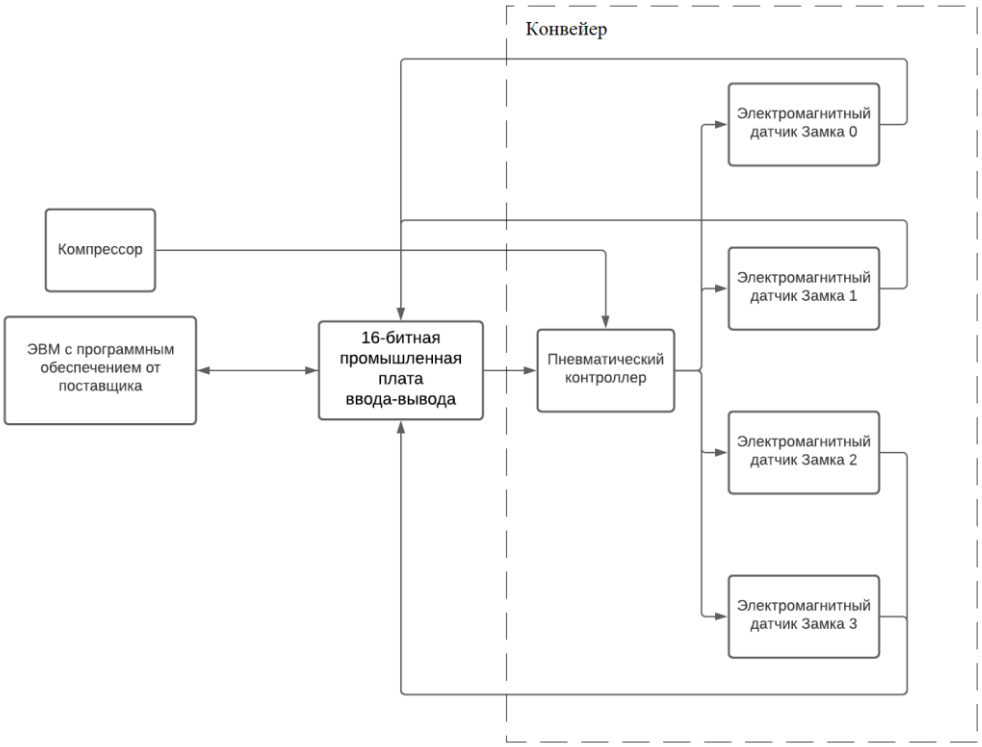


Рисунок 4. Структурная схема кольцевого ленточного конвейера

Данное программное решение имеет ряд существенных недостатков:

1) *Отсутствие режима автономной работы*;
2) *Отсутствие возможности подключения к облачной платформе «Yandex Cloud»*;
3) *Закрытый исходный код, не позволяющий вносить изменения*;

4) *Ограниченный функционал (невозможно открыть больше одного замка одновременно)*;

Исходя из данных недостатков можно сформулировать требования к функционалу программной части сервера устройства для управления конвейером:

1) *Программа должна поддерживать автономный режим работы*;
2) *Программа должна свободно взаимодействовать с облачной патформой «Yandex Cloud»*;
3) *Программа должна иметь открытый исходный код, позволяющий другим пользователям изменять его в зависимости от их целей*;
4) *Программа должна поддерживать управление всеми замками одновременно*;

3. Взаимодействие сервер устройства с кольцевым ленточным конвейером.

Так как у используемого оборудования отсутствует возможность подключения к интернету, взаимодействие промышленного оборудования с облачной платформой «Yandex Cloud» осуществляется через сервер устройства. В качестве сервера устройства выступает одноплатный компьютер Raspberry Pi 3 model B. Подключение ленточного кольцевого конвейра к серверу устройства (Raspberri Pi 3 model B) осуществляется «вразрез» (рисунок 5) между 16-битной промышленной платой ввода-ввывода и пневматическим контроллером. Поскольку напряжение логических уровней конвейера - 24В, а напряжение логических уровней Raspberry Pi- 3.3В, то используется плата оптической развязки, которая подключается к Raspberry Pi через кабель D25.

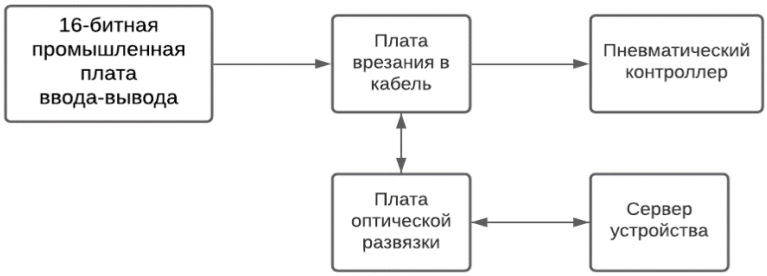


Рисунок 5. Схема подключения сервера устройства к конвейеру

На плате Raspberry существует 2 режима обозначения пинов BOARD и BCM, отличие лишь в том, что BOARD - указывает, что вы обращаетесь к контактам по номеру контакта штекера, а BCM - цифры после «GPIO» (рисунок 6). Назначения каждого входа-выхода указаны в таблице 1.

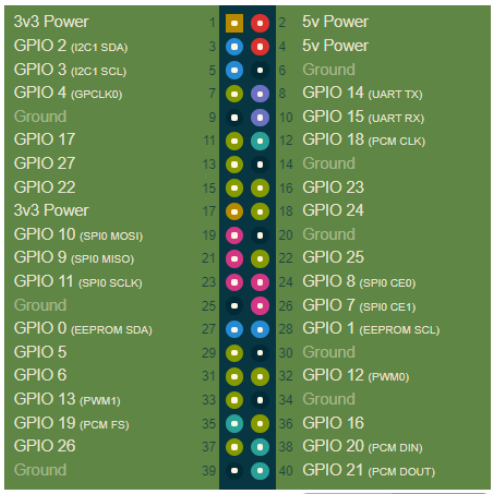


Рисунок 6. Цоколёвка Raspberry

Pin(аппар.)	BCM #	Назначение	Направление
7	4	Датчик 0	Вход
11	17	Датчик 0	Вход
12	18	Замок 0	Выход
13	27	Датчик 0	Вход
15	22	Датчик 0	Вход
16	23	Замок 0	Выход
18	24	Замок 0	Выход
22	25	Замок 0	Выход
26	7	E-Stop	Выход

Таблица 1. Схема подключения конвейера к Raspbery Pi

```
GPIO.setmode(GPIO.BCM)
lock=(18, 23, 24, 25)
sensor=(4, 17, 27, 22)
Estop = 7
interrupt = 1
sleep_in_lock = 5

test_voltage = (26, 19, 20, 16)

GPIO.setup([lock[0], lock[1], lock[2], lock[3]], GPIO.OUT)
GPIO.setup([sensor[0], sensor[1], sensor[2], sensor[3]], GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
GPIO.setup([test_voltage[0], test_voltage[1], test_voltage[2], test_voltage[3]], GPIO.OUT)

GPIO.output([test_voltage[0], test_voltage[1], test_voltage[2], test_voltage[3]], GPIO.HIGH)
```

Рисунок 7. Инициализация режима обозначения цоколёвки на языке Python

4. Взаимодействие кольцевого ленточного конвейера с облачной платформой «Yandex Cloud».

Основными элементами облачной системы «Yandex Cloud» являются устройство и реестр, которые могут обмениваться различными данными и командами с помощью MQTT-брокера. Обмен данными происходит по протоколу MQTT версии 3.1.1. Это упрощенный сетевой протокол для взаимодействия между устройствами по принципу издатель-подписчик. Чтобы начать обмен данными и командами между устройствами и реестрами, необходимо авторизоваться с помощью X.509-сертификатов или по логину и паролю. В данном курсовом проекте процесс авторизации выполняется при помощи X.509-сертификатов.

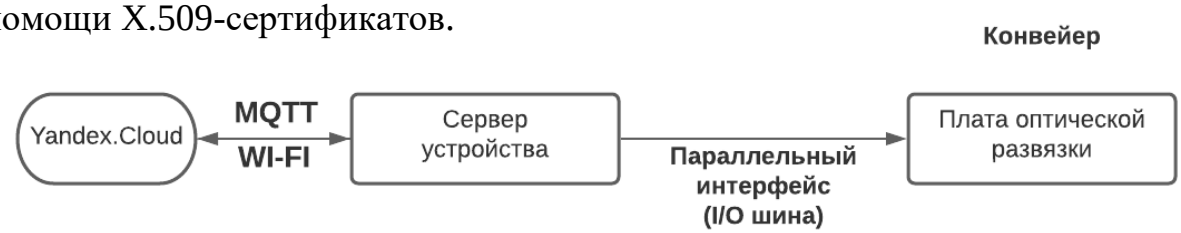


Рисунок 7. Схема взаимодействия сервера устройства с облачной системой.

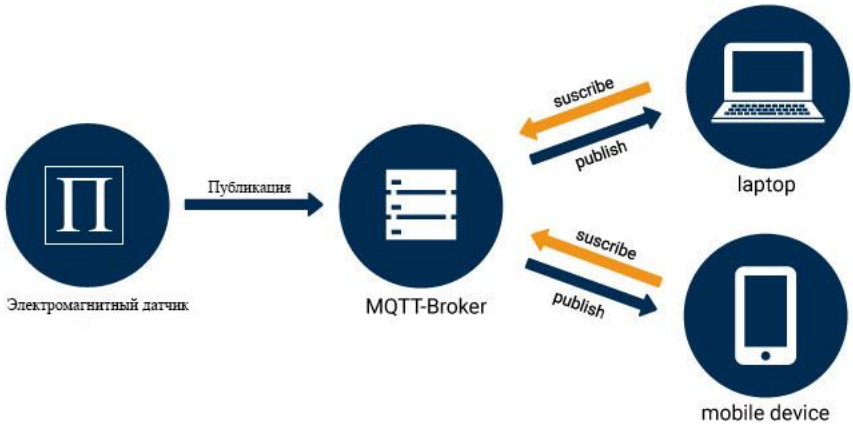


Рисунок 8. Схема работы протокола обмена

5 Разработка программной части сервера устройства.

Прежде чем приступать к написанию программной части сервера устройства, необходимо разработать алгоритмы работы основных функций программы. К числу основных функций программы можно отнести следующие:

1) *валидация полученных команд от облачной платформы*;

2) *открытие любого замка для прохождения паллеты через него*;

3) *4 функции, которые обрабатывают логику работы каждого замка*;

В данном курсовом проекте рассматривается 2 варианта обработки заготовки: 1) последовательный – заготовка вначале обрабатывается на первом станочном модуле затем на втором, 2) наоборот. Склад хранит информацию о том - какая заготовка каким способом будет обрабатываться, и при постановке паллеты на конвейер публикует сообщение в соответствующий топик. Поэтому заведем двумерный список array, в котором будет хранится информации о том какой тип обработки у заготовки и сколько кругов она прошла. Список выглядит следующим образом: [[1,0],[0,0], …]. Как только паллета достигает конкретного Замка, публикуется сообщение в определенный топик – так другие модули понимают, что могут приступить к выполнению своих задач. При линейном выполнении кода невозможно одновременно открывать сразу несколько замков, опубликовывать и анализировать соответствующие сообщения. Решением данной проблемы – является использование многопоточности, поэтому логика работы каждого замка – независимая единица и будет рассмотрена отдельно.

5.1 Использование многопоточности для одновременного управления замками кольцевого ленточного конвейера.

Многопоточность - процесс приложения разбит на потоки, которые параллельно - в одну единицу времени - обрабатываются процессором. Вычислительная нагрузка распределяется между двумя или более ядрами, так что интерфейс и другие компоненты программы не замедляют работу друг друга.

Стандартная библиотека Python предоставляет библиотеку threading, которая содержит необходимые классы для работы с потоками. Метод Thread – инициализирует поток, в качестве аргументов принимает функцию, и аргументы этой функции (рисунок 9). После инициализации для каждого

потока устанавливается флаг daemon (демон) = True. Это необходимо для корректного завершения программы - при завершении работы main потока все демонические потоки будут принудительно завершены. Метод start – просто запускает данный поток.

```
x0 = threading.Thread(target=check_lock0, args=(lock[0], sensor[0], 0, my_conv))
x0.daemon = True
x1 = threading.Thread(target=check_lock1, args=(lock[1], sensor[1], 1, my_conv))
x1.daemon = True
x2 = threading.Thread(target=check_lock2, args=(lock[2], sensor[2], 3, my_conv))
x2.daemon = True
x3 = threading.Thread(target=check_lock3, args=(lock[3], sensor[3], 3, my_conv))
x3.daemon = True
x0.start()
x1.start()
x2.start()
x3.start()
```

Рисунок 9. Часть кода, отвечающая инициализацию потоков

5.2 Валидация полученных команд от облачной платформы.

Событийное программирование фокусируется на событиях, поток программы напрямую зависит от данных событий. Модель, имеющая концепцию управляемого событиями программирования, называется асинхронной моделью. Программирование, управляемое событиями, зависит от цикла событий, который всегда прослушивает новые входящие события. Работа программирования, управляемого событиями, зависит от событий. Как только событие зацикливается, события решают, что и в каком порядке выполнять.

Рисунок 10. Блок-схема событийного программирования

```python
def handler(lock, sensor, Estop):

    conv = mqtt.Client(client_id="are6c1grj2ojp532jr3u")
    my_conv = Conveyer(conv, lock, sensor, Estop)

    x0 = threading.Thread(target=check_lock0, args=(lock[0], sensor[0], 0, my_conv))
    x0.daemon = True
    x1 = threading.Thread(target=check_lock1, args=(lock[1], sensor[1], 1, my_conv))
    x1.daemon = True
    x2 = threading.Thread(target=check_lock2, args=(lock[2], sensor[2], 3, my_conv))
    x2.daemon = True
    x3 = threading.Thread(target=check_lock3, args=(lock[3], sensor[3], 3, my_conv))
    x3.daemon = True
    x0.start()
    x1.start()
    x2.start()
    x3.start()
    conv.on_connect = Conveyer.on_connect
    conv.on_message = my_conv.on_message
    conv.on_publish = Conveyer.on_publish

    conv.tls_set(ca_certs="/home/pi/Desktop/rootCA.crt", certfile="/home/pi/Desktop/cert_conveyer.pem",
                 keyfile="/home/pi/Desktop/key_conveyer.pem", cert_reqs=ssl.CERT_REQUIRED,
                 tls_version=ssl.PROTOCOL_TLSv1_2)
    conv.connect("mqtt.cloud.yandex.net", port=8883)

    conv.loop_forever()
```

Рисунок 11. Реализация функции handler на языке Python

Функция handler обрабатывает 3 события «on_connect» – момент подключения к системе диспетчеризации (отправляет сообщение о своей готовности), «on_message» – получение определенного сообщения в соответствующий топик, «on_publish» – публикация сообщения в соответствующий топик. Данные «события» реализованы внутри методов класса Conveyer (рисунок 12).

```python
@staticmethod
def on_connect(client, userdata, flags, rc):
    print("Connected: " + str(rc))
    client.subscribe("$devices/are6c1grj2ojp532jr3u/commands", qos=1)
    print("Subscribed")

def on_message(self, client, userdata, message):
    print(message.topic + ' ' + str(message.payload))
    if str(message.payload) == "b'processing_mode1'":
        self.array.append([1,0])
    if str(message.payload) == "b'processing_mode0'":
        self.array.append([0,0])
    if str(message.payload) == "b'machine1_done'":
        unlock(self.lock[0], 0)
        self.skip_counter_0 += 1
    if str(message.payload) == "b'machine2_done'":
        unlock(self.lock[1], 1)
        self.skip_counter_0 += 1
    if str(message.payload) == "b'quality_control_done'":
        unlock(self.lock[2], 2)
    if str(message.payload) == "b'stop'":
        GPIO.output(Estop, GPIO.HIGH)
        sleep(10)
        GPIO.cleanup()
        sys.exit()
    if str(message.payload) == "b'state'":
        self.publisher.publish("$devices/are6c1grj2ojp532jr3u/events", payload="ready", qos=1)
    elif str(message.payload) == "b'process'":
        print("Conveyer processing command")
@staticmethod
def on_publish(client, userdata, mid):
    print("Message published   ")
```

Рисунок 12. Команды, которые могут быть получены от облачной системы «Yandex Cloud»

5.3 Проектирование функции, обрабатывающей логику открытия любого замка.

Функция, обеспечивающая прохождение паллеты через замок. На соответствующем выводе устанавливается логическая единица, и дальнейший код выполняется с задержкой «sleep_in_lock». В Python есть встроенная поддержка для погружения программы в сон. У модуля time есть функция sleep(), что позволяет отсрочить выполнение вызываемого потока на указанное количество секунд. После указанной задержки на соответствующем выводе устанавливается логический ноль.

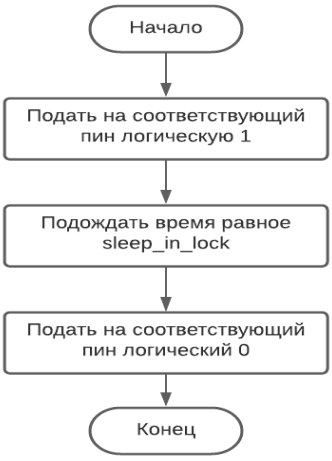


Рисунок 13. Блок-схема данной функции.

```
def unlock(lock, i):
    GPIO.output(lock, GPIO.HIGH)
    sleep(sleep_in_lock)
    GPIO.output(lock, GPIO.LOW)
```

Рисунок 14. Реализация на языке Python

5.4 Проектирование функции, обрабатывающей логику работы Замка 0.

После того как штабеллер опускает паллету на конвейер, в массив array добавляется информация о том, как необходимо обработать данную заготовку. Паллета перемещается к Замку 0, если данную заготовку необходимо обработать сначала на первом станке – публикуется сообщение о том, что заготовка «ready», тем самым токарный станочный модуль понимает, что может приступить к выполнению своей задачи. Далее ожидается сообщение от этого модуля в соответствующий топик о том, что заготовка готова покинуть Замок 0. Если же заготовка обрабатывается сначала на втором станочном модуле, Замок 0 открывается. Так же есть

переменные «skip_counter_i», (где i = 0, 1, 3) которые увеличиваются после прохождения паллетой замков к которым они относятся, они необходимы для того, чтобы понимать какую заготовку в каком порядке обрабатывать.

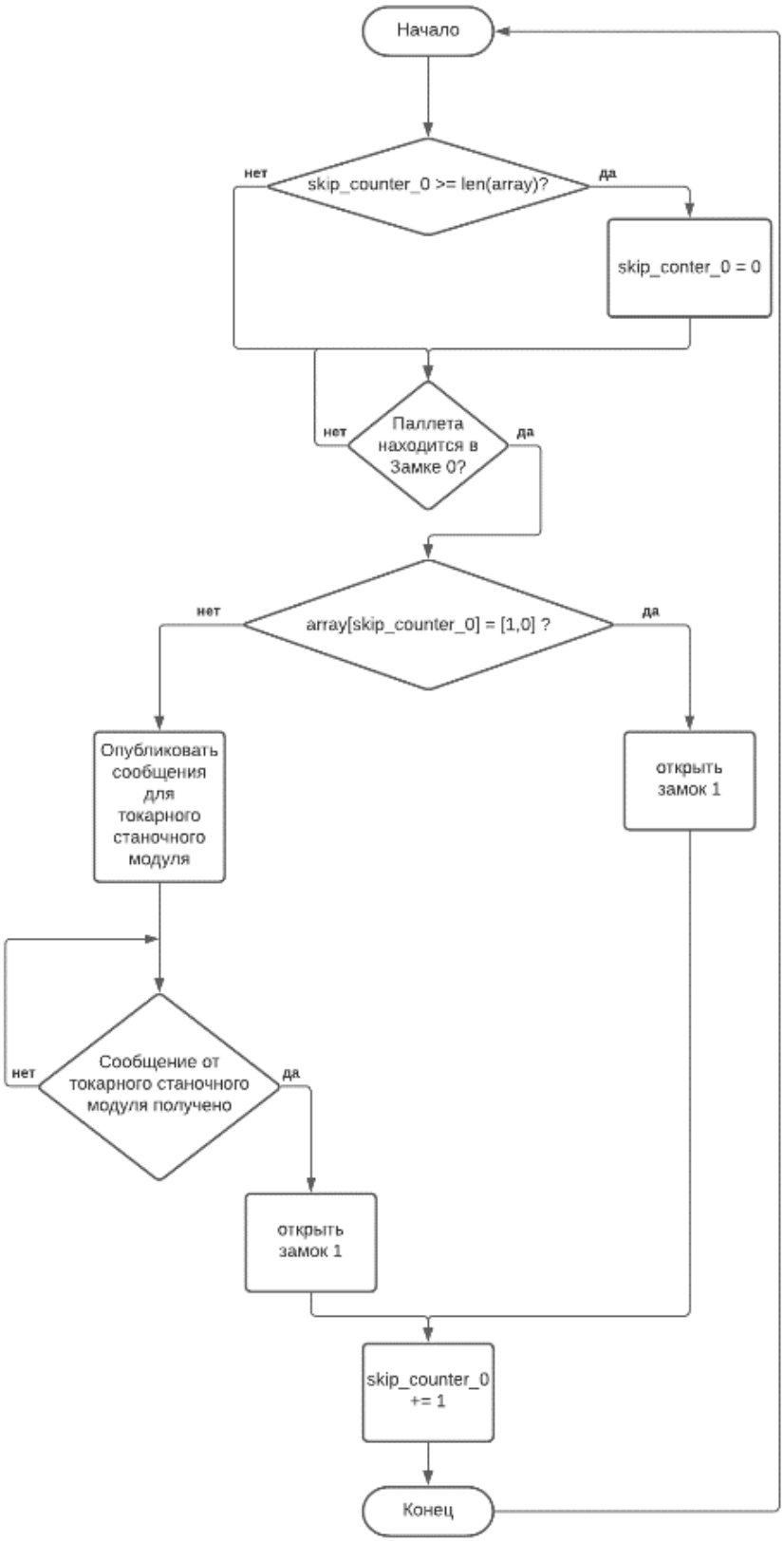


Рисунок 15. Блок-схема: логика работы Замка 0

```python
def check_lock0(lock, sensor, i, conv):
    while interrupt:
        if conv.skip_counter_0 >= len(conv.array):
            conv.skip_counter_0 = 0
        flag = 0
        while GPIO.input(sensor):
            for elem in conv.array[conv.skip_counter_0:]:
                if elem[0] == 1 and elem[1] == 0:
                    conv.skip_counter_0 += 1
                    unlock(lock, i)
                    sleep(1)
                else:
                    if flag == 0:
                        conv.publisher.publish("$devices/are6c1grj2ojp532jr3u/events/done", payload="ready", qos=1)
                        flag = 1
                break
        sleep(1)
```

Рисунок 16. Реализация на языке Python

5.5 Проектирование функции, обрабатывающей логику работы Замка 1.

При достижении паллетой Замка 1, осуществляется проверка на то какой тип обработки у данной паллеты и сколько кругов она прошла. Если же паллета прошла круг и должна обрабатываться сначала на втором станке Замок 1 открывается и паллета следует дальше, в другом случае публикуется в соответствующий топик информация для станочного модуля 2, что он может приступить к выполнению своих задач. Как только получено сообщение о том, что заготовка может покинуть Замок 1 от станочного модуля 2 – Замок 1. При прохождении паллеты через Замок 1 «skip_counter_1» увеличивается.

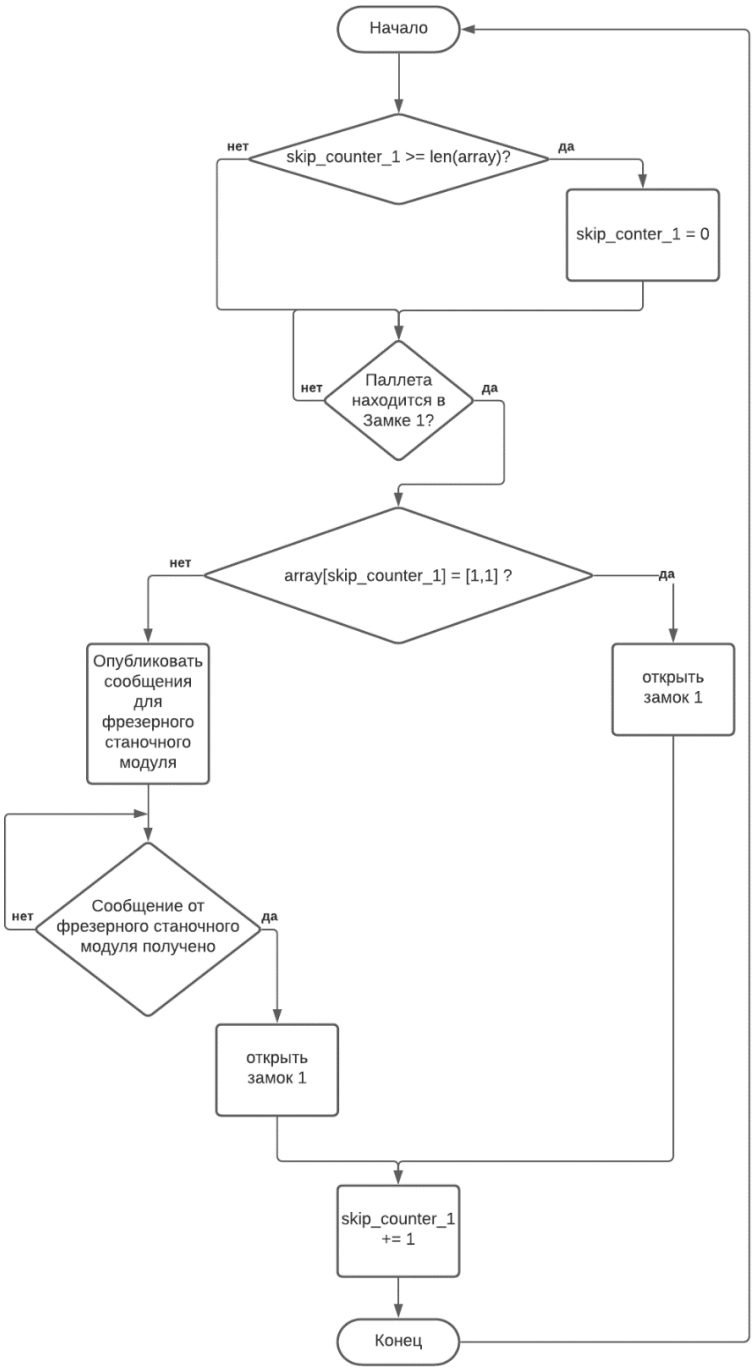


Рисунок 17. Блок-схема: логика работы Замка 1

```python
def check_lock1(lock, sensor, i, conv):
    while interrupt:
        if conv.skip_counter_1 >= len(conv.array):
            conv.skip_counter_1 = 0
        flag = 0
        while GPIO.input(sensor):
            for elem in conv.array[conv.skip_counter_1:]:
                if elem[0] == 1 and elem[1] == 1:
                    conv.skip_counter_1 += 1
                    unlock(lock, i)
                    sleep(1)
                else:
                    if flag == 0:
                        conv.publisher.publish("$devices/are6c1grj2ojp532jr3u/events/done", payload="ready", qos=1)
                        flag = 1
                break
        sleep(1)
```

Рисунок 18. Реализация на языке Python

5.6 Проектирование функции, обрабатывающей логику работы Замка 2

Как только паллета оказывается в Замке 2, публикуется сообщения для модуля контроля качества, паллета находится в Замке 2, пока от модуля не поступит сообщения о том, что он завершил свои задачи. В случае получения данного сообщения открывается Замок 2 и паллета следует дальше.

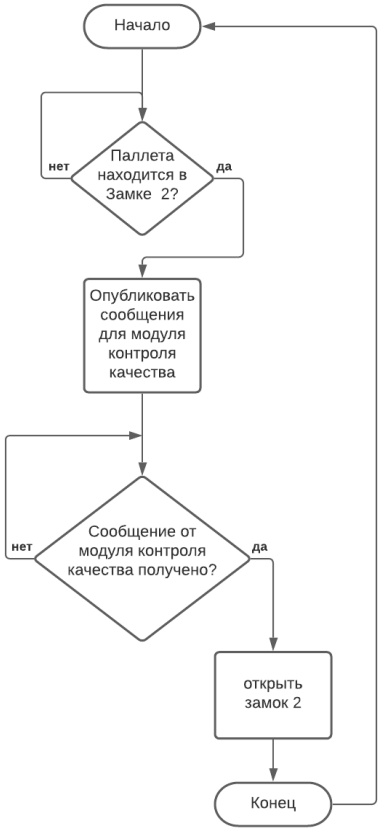


Рисунок 19. Блок-схема: логика работы Замка 2

```python
def check_lock2(lock, sensor, i, conv):
    while interrupt:
        flag = 0
        while GPIO.input(sensor):
            if flag == 0:
            conv.publisher.publish("$devices/are28c1grj2ojp532jr3u/events/done", payload="ready", qos=1)
            flag = 1
        sleep(1)
```

Рисунок 20. Реализация на языке Python

5.7 Проектирование функции, обрабатывающей логику работы Замка 3

Как только паллета достигает Замка 3, начинается проверка – сколько кругов пройдено данной паллетой и какой режим обработки у неё. Если у паллеты был первый вариант обработки, она отправляется на склад – отправляется сообщение для склада и штабеллер забирает заготовку, если второй вариант обработки – открывается Замок 3, паллета отправляется на второй круг и в array[skip_counter_3][1] записывается единица.

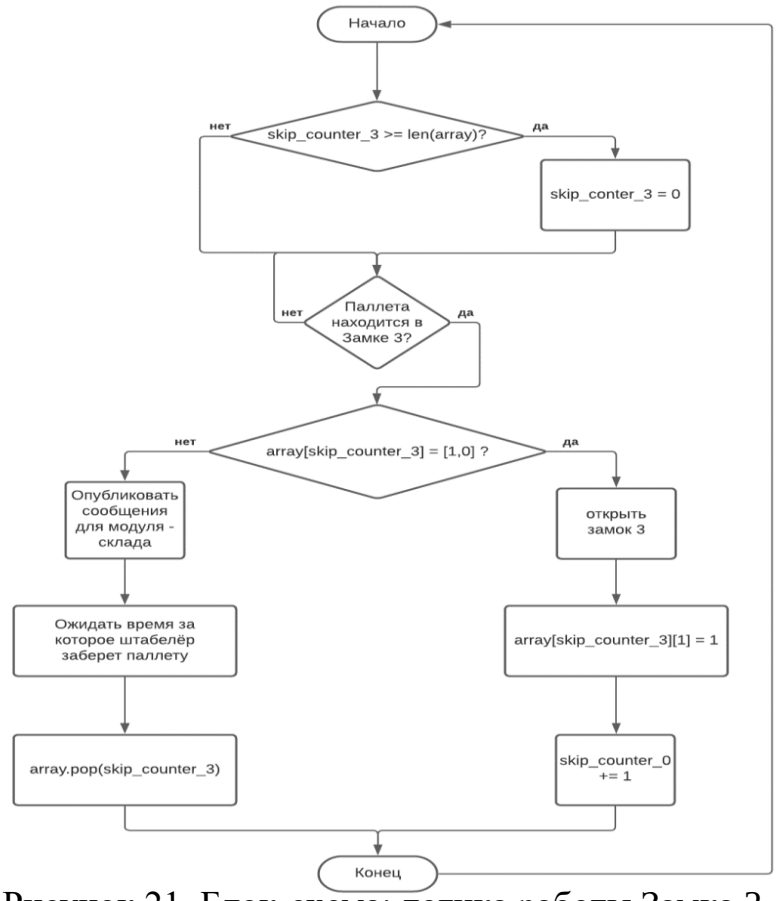


Рисунок 21. Блок-схема: логика работы Замка 3.

```python
def check_lock3(lock, sensor, i, conv):
    while interrupt:
        flag = 0
        if conv.skip_counter_3 >= len(conv.array):
            conv.skip_counter_3 = 0
        while GPIO.input(sensor):
            for elem in conv.array[conv.skip_counter_3:]:
                if elem[0] == 1 and elem[1] == 0:
                    conv.array[conv.skip_counter_3][1] = 1
                    conv.skip_counter_3 += 1
                    unlock(lock, i)
                    sleep(1)
                else:
                    if flag == 0:
                        conv.publisher.publish("$devices/are6c1grj2ojp532jr3u/events/done", payload="ready", qos=1)
                        conv.array.pop(conv.skip_counter_3)
                        flag = 1
                        sleep(20)
                break
        sleep(1)
```

Рисунок 22. Реализация на языке Python

6 Аварийная остановка.

Аварийная остановка производственной линии осуществляется в соответствии с планом ликвидации аварий, разработанном на производстве и утвержденном заместителем Генерального директора по производству и техническим вопросам предприятия. В случае аварийной остановки - от облачной системы поступает соответствующая команда - на соответствующем выводе «E-stop» сервера устройства устанавливается логическая единица и кольцевой ленточный конвейер останавливается.

Заключение

На основании проделанной работы были получены следующие результаты:

1) Проанализирована существующая ГПС и выявлены недостатки её программного решения.

2) Исходя из данных недостатков были сформулированы требования к функционалу программной части сервера-устройства для управления кольцевым ленточным конвейером.

3) Были разработаны алгоритмы: функции валидации полученных команд от облачной платформы «Yandex Cloud», функция открытия любого замка для прохождения паллеты через него и 4 функции, которые обрабатывают логику работы каждого замка;

4) Была реализована программная часть сервера устройства на языке Python.

В результате данного курсового проекта была разработана программная часть сервера устройства для облачной системы управления кольцевым конвейером на основе облачной платформы «Yandex Cloud», которая демонстрирует работу концепции промышленного интернета вещей.

Список используемой литературы

1. McEwen A., Cassimally H. Designing the Internet of Things. West Sussex, United Kingdom: John Wiley and Sons, 2014. – 276 p.
2. Выжигин А.Ю. Гибкие производственные системы: учебное пособие: Машиностроение, 2012. – 288 с.
3. Технические средства автоматизации и управления на основе IIoT/IoT : учебное пособие / Ю. П. Страшун. - Санкт-Петербург : Лань, 2020. - 176 с.
4. Raspberry Pi GPIO Pinout [Электронный ресурс] // Pinout [сайт]. [2020] URL: https://pinout.xyz/ (дата обращения 25.10.2020)
5. Getting started with Raspberry Pi [Электронный ресурс] // Raspberrypi [сайт]. [2020] URL: https://projects.raspberrypi.org/en/projects/raspberry-pi-getting-started (дата обращения 10.10.2020)
6. Златопольский Д.М. Основы программирования на языке Python. – М.: ДМК Пресс, 2017. – 284 с.
7. Конвейеры: Справочник / Р. А. Волков, А. Н, Гнутов, В. К. Дьячков и др, Под общ. ред. Ю. А. Пертена. Л.; Машиностроение, Ленингр, отд-ние, 1984. 367 с.
8. Овсянников, М.В. Организация индустриального интернета на основе SAP HANA Cloud Platform: учебное пособие 0 / М.В. Овсянников, С.А. Буханов, С.А. Подкопаев. – М.: издательство МГТУ им. Н. Э. Баумана, 2019. – 195 с.
9. Пошаговые инструкции «Yandex Cloud» – Документация [Электронный ресурс] // Cloud.Yandex [сайт]. [2020] URL: https://cloud.yandex.ru/docs/iotcore/operations/publish (дата обращения 20.11.2020)
10. [Электронный ресурс] // Pythontutor [сайт]. [2020]. URL: https://pythontutor.ru/ (дата обращения 17.10.2020)

Приложение 1

Реализация функции обработки и выполнения задания для конвейера на языке Python[4].

```
import paho.mqtt.client as mqtt
import ssl
from time import sleep
import RPi.GPIO as GPIO
import signal
import sys
import threading

class Conveyer:

    def __init__(self, conv, lock, sensor, Estop):
        self.publisher = conv
        self.lock = lock
        self.sensor = sensor
        self.array = []
        self.skip_counter_0 = 0
        self.skip_counter_1 = 0
        self.skip_counter_3 = 0

    @staticmethod
    def on_connect(client, userdata, flags, rc):
        print("Connected: " + str(rc))
        client.subscribe("$devices/are6c1grj2ojp532jr3u/commands", qos=1)
        print("Subscribed")

    def on_message(self, client, userdata, message):
        print(message.topic + ' ' + str(message.payload))
```

```
        if str(message.payload) == "b'processing_mode1'":
            self.array.append([1,0])
        if str(message.payload) == "b'processing_mode0'":
            self.array.append([0,0])
        if str(message.payload) == "b'machine1_done'":
            unlock(self.lock[0], 0)
            self.skip_counter_0 += 1
        if str(message.payload) == "b'machine2_done'":
            unlock(self.lock[1], 1)
            self.skip_counter_0 += 1
        if str(message.payload) == "b'quality_control_done'":
            unlock(self.lock[2], 2)
        if str(message.payload) == "b'stop'":
            GPIO.output(Estop, GPIO.HIGH)
            sleep(10)
            GPIO.cleanup()
            sys.exit()
        if str(message.payload) == "b'state'":
            self.publisher.publish("$devices/are6c1grj2ojp532jr3u/events",
payload="ready", qos=1)
        elif str(message.payload) == "b'process'":
            print("Conveyer processing command")
    @staticmethod
    def on_publish(client, userdata, mid):
        print("Message published   ")

def handler(lock, sensor, Estop):

    conv = mqtt.Client(client_id="are6c1grj2ojp532jr3u")
    my_conv = Conveyer(conv, lock, sensor, Estop)
```

```
    x0 = threading.Thread(target=check_lock0, args=(lock[0], sensor[0], 0,
my_conv))
    x0.daemon = True
    x1 = threading.Thread(target=check_lock1, args=(lock[1], sensor[1], 1,
my_conv))
    x1.daemon = True
    x2 = threading.Thread(target=check_lock2, args=(lock[2], sensor[2], 3,
my_conv))
    x2.daemon = True
    x3 = threading.Thread(target=check_lock3, args=(lock[3], sensor[3], 3,
my_conv))
    x3.daemon = True
    x0.start()
    x1.start()
    x2.start()
    x3.start()
    conv.on_connect = Conveyer.on_connect
    conv.on_message = my_conv.on_message
    conv.on_publish = Conveyer.on_publish

    conv.tls_set(ca_certs="/home/pi/Desktop/rootCA.crt",
certfile="/home/pi/Desktop/cert_conveyer.pem",
                 keyfile="/home/pi/Desktop/key_conveyer.pem",
cert_reqs=ssl.CERT_REQUIRED,
                 tls_version=ssl.PROTOCOL_TLSv1_2)
    conv.connect("mqtt.cloud.yandex.net", port=8883)

    conv.loop_forever()
```

```
def check_lock0(lock, sensor, i, conv):
    while interrupt:
        if conv.skip_counter_0 >= len(conv.array):
            conv.skip_counter_0 = 0
        flag = 0
        while GPIO.input(sensor):
            for elem in conv.array[conv.skip_counter_0:]:
                if elem[0] == 1 and elem[1] == 0:
                    conv.skip_counter_0 += 1
                    unlock(lock, i)
                    sleep(1)
                else:
                    if flag == 0:

conv.publisher.publish("$devices/are6c1grj2ojp532jr3u/events/done",
payload="ready", qos=1)
                        flag = 1
                break
        sleep(1)

def check_lock1(lock, sensor, i, conv):
    while interrupt:
        if conv.skip_counter_1 >= len(conv.array):
            conv.skip_counter_1 = 0
        flag = 0
        while GPIO.input(sensor):
            for elem in conv.array[conv.skip_counter_1:]:
                if elem[0] == 1 and elem[1] == 1:
                    conv.skip_counter_1 += 1
```

```
                unlock(lock, i)
                sleep(1)
            else:
                if flag == 0:

conv.publisher.publish("$devices/are6c1grj2ojp532jr3u/events/done",
payload="ready", qos=1)
                    flag = 1
            break
        sleep(1)

def check_lock2(lock, sensor, i, conv):
    while interrupt:
        flag = 0
        while GPIO.input(sensor):
            if flag == 0:
            conv.publisher.publish("$devices/are28c1grj2ojp532jr3u/events/done",
payload="ready", qos=1)
            flag = 1
        sleep(1)

def check_lock3(lock, sensor, i, conv):
    while interrupt:
        flag = 0
        if conv.skip_counter_3 >= len(conv.array):
            conv.skip_counter_3 = 0
        while GPIO.input(sensor):
            for elem in conv.array[conv.skip_counter_3:]:
                if elem[0] == 1 and elem[1] == 0:
                    conv.array[conv.skip_counter_3][1] = 1
```

```
                conv.skip_counter_3 += 1
                unlock(lock, i)
                sleep(1)
            else:
                if flag == 0:

conv.publisher.publish("$devices/are6c1grj2ojp532jr3u/events/done",
payload="ready", qos=1)
                    conv.array.pop(conv.skip_counter_3)
                    flag = 1
                    sleep(20)
            break
        sleep(1)

def unlock(lock, i):
    GPIO.output(lock, GPIO.HIGH)
    sleep(sleep_in_lock)
    GPIO.output(lock, GPIO.LOW)



def unlock2(lock, sensor, i):
    while interrupt:
        if GPIO.input(sensor):
            GPIO.output(lock, GPIO.HIGH)
            sleep(sleep_in_lock)
            GPIO.output(lock, GPIO.LOW)
        sleep(0.5)



def signal_handler(sig, frame):
```

```
    global interrupt
    interrupt = 1
    GPIO.cleanup()
    sys.exit()


if __name__ == "__main__":
    GPIO.setmode(GPIO.BCM)
    lock=(18, 23, 24, 25)
    sensor=(4, 17, 27, 22)
    Estop = 7
    interrupt = 1
    sleep_in_lock = 5

    test_voltage = (26, 19, 20, 16)

    GPIO.setup([lock[0], lock[1], lock[2], lock[3]], GPIO.OUT)
    GPIO.setup([sensor[0], sensor[1], sensor[2], sensor[3]], GPIO.IN,
pull_up_down=GPIO.PUD_DOWN)
    GPIO.setup([test_voltage[0], test_voltage[1], test_voltage[2], test_voltage[3]],
GPIO.OUT)

    GPIO.output([test_voltage[0], test_voltage[1], test_voltage[2], test_voltage[3]],
GPIO.HIGH)
    signal.signal(signal.SIGINT, signal_handler)

    handler(lock, sensor, Estop)
```

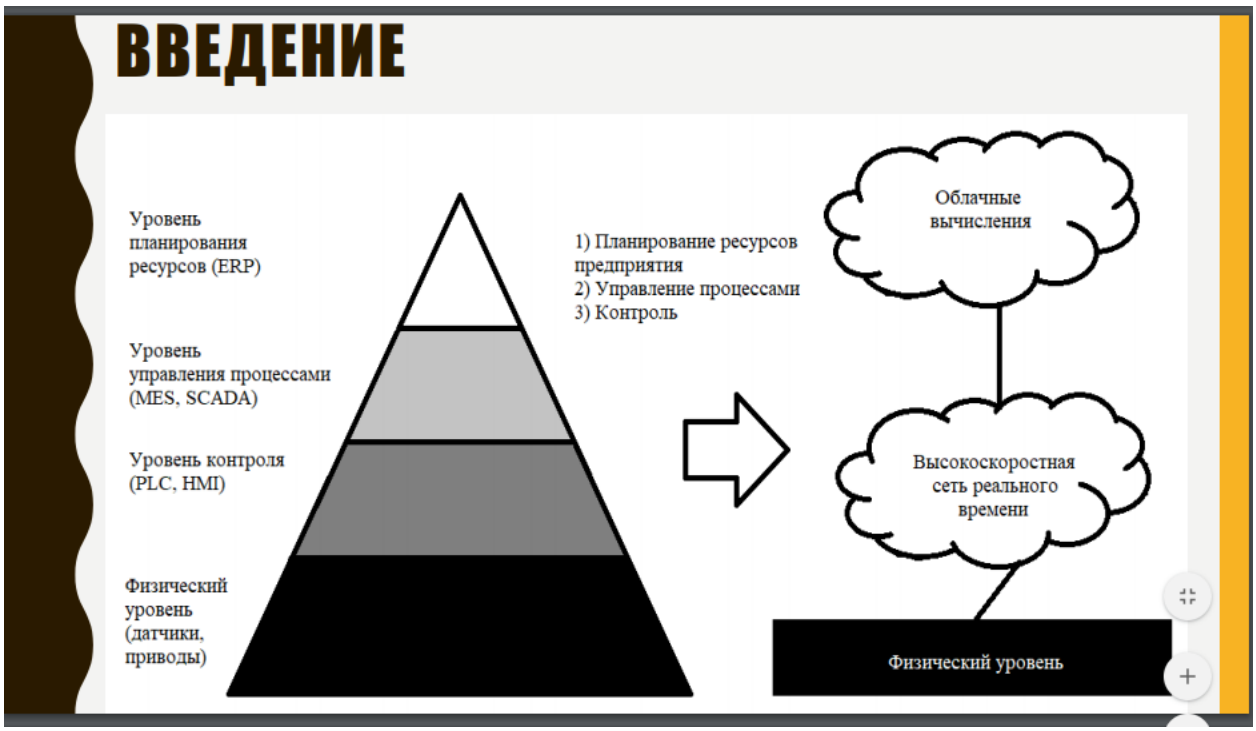
Приложение 2

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

Курсовой проект на тему:

«Проектирование программной части сервера устройства для облачной системы управления кольцевым конвейером на основе облачной платформы «Yandex Cloud»

Выполнил студент группы РК9-74
Рындя Е. И.
Руководитель проекта
Буханов С. А.



ЦЕЛЬ И ЗАДАЧИ КУРСОВОГО ПРОЕКТА

Цель курсового проекта – создание программной части сервера устройства для облачной системы управления кольцевым конвейером на основе облачной платформы «Yandex Cloud».

Задачи курсового проекта:

1) Проанализировать существующее программное обеспечение, реализованное Denford, и выявить его недостатки.

2) На основе проанализированных данных сформулировать требования, относящиеся к проектируемой программной части сервера устройства.

3) Разработать алгоритмы управления конвейером Denford для получения им команд от облачной системы «Yandex Cloud».

4) Разработать программный код на языке Python для программной части сервера устройства.

АНАЛИЗ ГИБКОЙ ПРОИЗВОДСТВЕННОЙ СИСТЕМЫ «DENFORD».

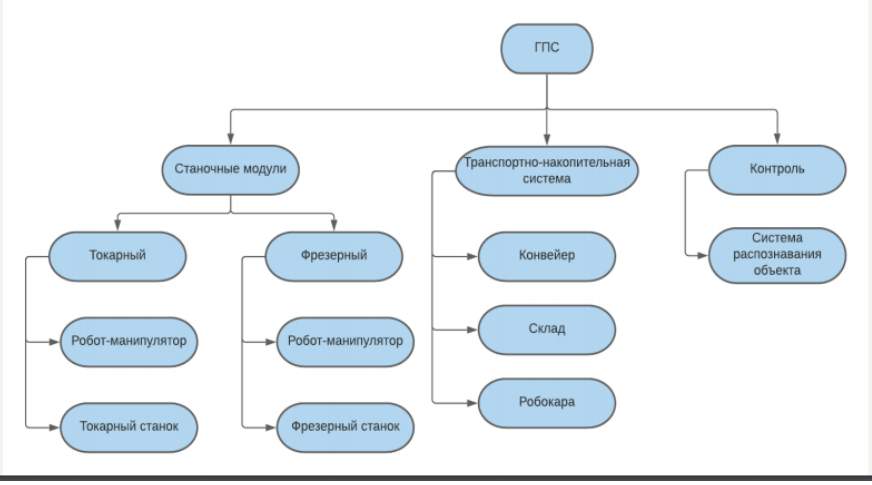
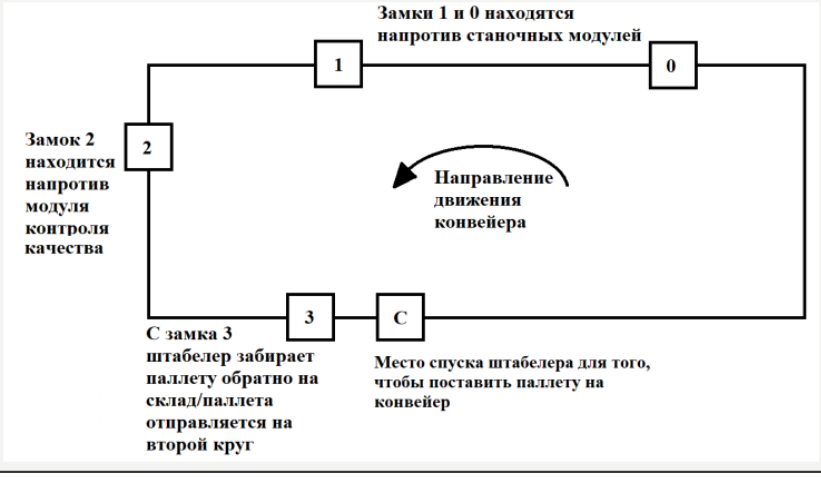


СХЕМА РАСПОЛОЖЕНИЯ ЗАМКОВ



СТРУКТУРНАЯ СХЕМА КОЛЬЦЕВОГО ЛЕНТОЧНОГО КОНВЕЙЕРА

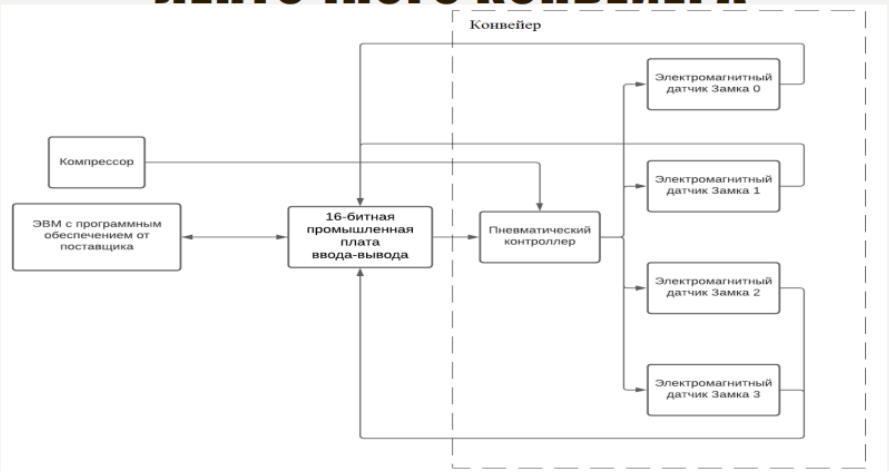
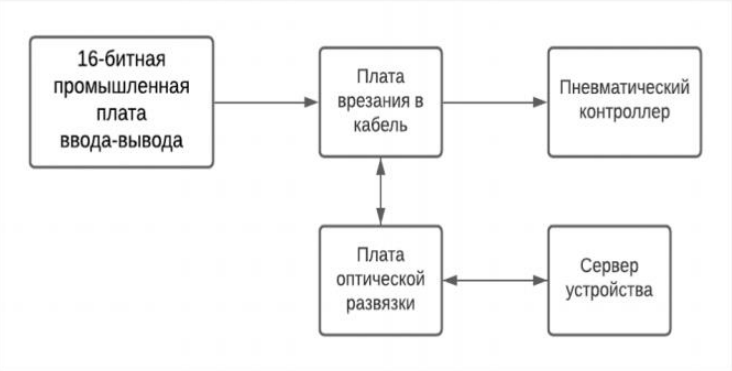


СХЕМА ПОДКЛЮЧЕНИЯ СЕРВЕРА УСТРОЙСТВА К КОНВЕЙЕРУ



ПОДКЛЮЧЕНИЕ КОНВЕЙЕРА К RASPBERRY PI

Pin(аппар.)	BCM #	Назначение	Направление
7	4	Датчик 0	Вход
11	17	Датчик 0	Вход
12	18	Замок 0	Выход
13	27	Датчик 0	Вход
15	22	Датчик 0	Вход
16	23	Замок 0	Выход
18	24	Замок 0	Выход
22	25	Замок 0	Выход
26	7	E-Stop	Выход

СХЕМА ВЗАИМОДЕЙСТВИЯ СЕРВЕРА УСТРОЙСТВА С ОБЛАЧНОЙ ПЛАТФОРМОЙ YANDEX CLOUD

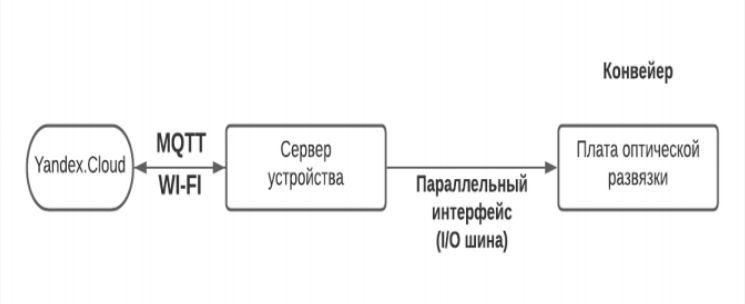
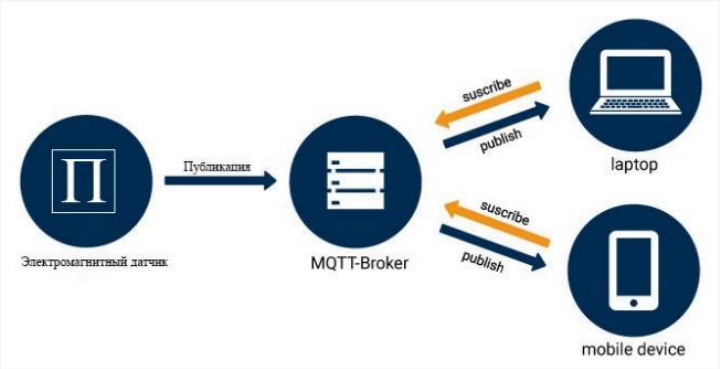
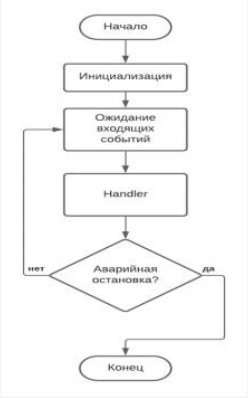


СХЕМА РАБОТЫ ПРОТОКОЛА ОБМЕНА





БЛОК СХЕМА СОБЫТИЙНОГО ПРОГРАММИРОВАНИЯ

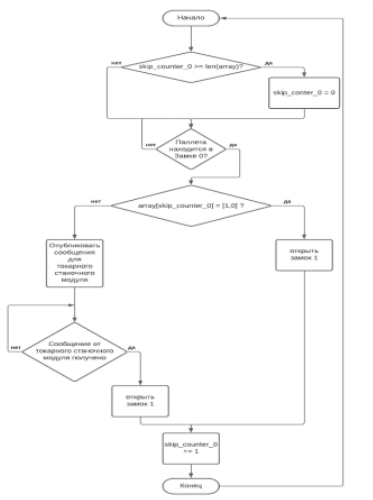
```python
def handler(lock, sensor, Estop):

    conv = mqtt.Client(client_id="are6c1grj2ojp532jr3u")
    my_conv = Conveyer(conv, lock, sensor, Estop)

    x0 = threading.Thread(target=check_lock0, args=(lock[0], sensor[0], 0, my_conv))
    x0.daemon = True
    x1 = threading.Thread(target=check_lock1, args=(lock[1], sensor[1], 1, my_conv))
    x1.daemon = True
    x2 = threading.Thread(target=check_lock2, args=(lock[2], sensor[2], 3, my_conv))
    x2.daemon = True
    x3 = threading.Thread(target=check_lock3, args=(lock[3], sensor[3], 3, my_conv))
    x3.daemon = True
    x0.start()
    x1.start()
    x2.start()
    x3.start()
    conv.on_connect = Conveyer.on_connect
    conv.on_message = my_conv.on_message
    conv.on_publish = Conveyer.on_publish

    conv.tls_set(ca_certs="/home/pi/Desktop/rootCA.crt", certfile="/home/pi/Desktop/cert_conveyer.pem",
                 keyfile="/home/pi/Desktop/key_conveyer.pem", cert_reqs=ssl.CERT_REQUIRED,
                 tls_version=ssl.PROTOCOL_TLSv1_2)
    conv.connect("mqtt.cloud.yandex.net", port=8883)

    conv.loop_forever()
```

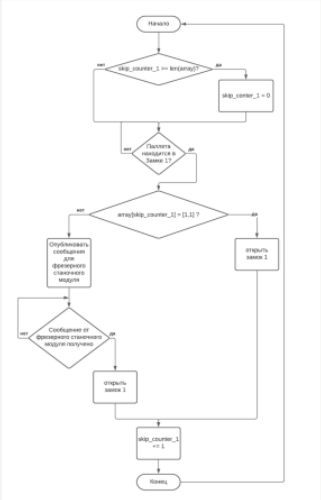
Реализация функции handler на языке Python

ПРОЕКТИРОВАНИЕ ФУНКЦИИ, ОБРАБАТЫВАЮЩЕЙ ЛОГИКУ ОТКРЫТИЯ ЛЮБОГО ЗАМКА

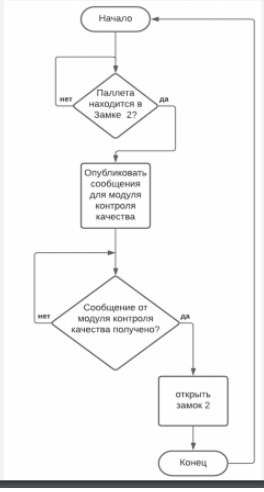
```python
def unlock(lock, i):
    GPIO.output(lock, GPIO.HIGH)
    sleep(sleep_in_lock)
    GPIO.output(lock, GPIO.LOW)
```

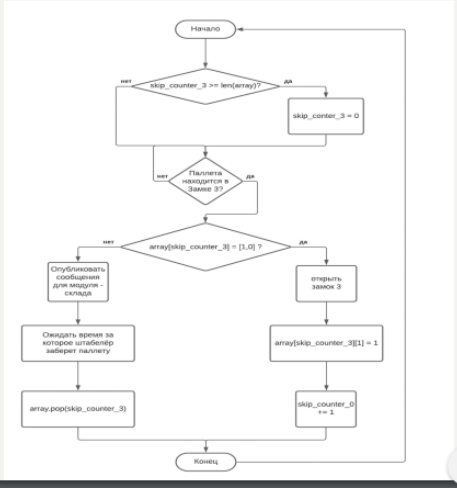
ПРОЕКТИРОВАНИЕ ФУНКЦИЙ, ОБРАБАТЫВАЮЩЕЙ ЛОГИКУ РАБОТЫ ЗАМКА 0 И 1





ПРОЕКТИРОВАНИЕ ФУНКЦИЙ, ОБРАБАТЫВАЮЩЕЙ ЛОГИКУ РАБОТЫ ЗАМКА 2 И 3





ЗАКЛЮЧЕНИЕ

На основании проделанной работы были получены следующие **результаты**:

1) Проанализирована существующая ГПС и выявлены недостатки её программного решения.

2) Исходя из данных недостатков были сформулированы требования к функционалу программной части сервера-устройства для управления кольцевым ленточным конвейером.

3) Были разработаны алгоритмы: функции валидации полученных команд от облачной платформы «Yandex Cloud», функция открытия любого замка для прохождения паллеты через него и 4 функции, которые обрабатывают логику работы каждого замка;

4) Была реализована программная часть сервера устройства на языке Python.

В результате данного курсового проекта была разработана программная часть сервера устройства для облачной системы управления кольцевым конвейером на основе облачной платформы «Yandex Cloud», которая демонстрирует работу концепции промышленного интернета вещей.